

Competitive Programming

Section 2: Simulation and brute force

Competitive Programming Course

2026-07-04

Table of contents

1	Where this section fits	3
2	Simulation	4
2.1	State diagram	4
2.2	Update order matters	4
3	Brute force	6
3.1	Example: two independent variables	6
3.2	Candidate count table	7
4	Exhaustive subsets	8
5	Simulation versus brute force	9
6	Bounds before loops	10
7	Proof habits for exhaustive search	11
8	Brute force as a testing oracle	12
9	C++ implementation notes	13
10	Python implementation notes	14
11	Common mistakes	15
12	Before you submit	16
13	Exercises	17

1 Where this section fits

The previous section trained the first contest reflex: read the constraints before choosing an algorithm. This section trains the next reflex:

If the statement describes a small process, simulate it exactly. If the search space is small, enumerate it deliberately.

Simulation and brute force are not beginner-only techniques. They appear at every level:

- as complete solutions for small constraints;
- as subroutines inside harder algorithms;
- as slow reference solutions for stress testing;
- as a way to discover patterns before proving them.

The difference between a weak brute force and a strong brute force is not the number of loops. It is whether the loops are justified by the constraints and whether the state is updated exactly.

2 Simulation

A **simulation** implements the process described by the statement step by step. The algorithm is usually obvious once the state is clear.

The central questions are:

1. What information describes the current state?
2. What event or command changes the state?
3. In what order do events occur?
4. What quantity must be reported after the process?

For a grid-walking robot, the state may be only (x, y) . For a game, the state may include a turn number, scores, cards, cooldown timers, and a queue of future events.

2.1 State diagram

For the command string URDL, the robot state evolves as:

```
start: (0, 0)
U:     (0, 1)
R:     (1, 1)
D:     (1, 0)
L:     (0, 0)
```

As arrows:

```
(0,0) --U--> (0,1) --R--> (1,1) --D--> (1,0) --L--> (0,0)
```

The final position is $(0, 0)$, but the maximum Manhattan distance reached is 2, at $(1, 1)$. If you only compute the final position, you miss information that existed during the simulation.

2.2 Update order matters

Suppose a problem asks:

after each move, record the maximum distance reached.

Then the update order is:

```
apply move
compute distance
update maximum
```

If the problem asked for the maximum distance before each move, the order would be different. Many wrong simulation submissions are one line away from correct: they update the right variables in the wrong order.

3 Brute force

Brute force means enumerating all candidates in a search space and checking each one directly.

The phrase is sometimes used dismissively, but in contests it is a precise tool. If there are only 40,000 candidates, brute force is often the cleanest solution. If there are 10^{12} candidates, brute force is not a solution unless the search space can be reduced.

The workflow is:

```
define candidate space
    |
    v
bound its size from constraints
    |
    v
loop over candidates
    |
    v
check each candidate exactly
    |
    v
keep the best or first valid candidate
```

The two important verbs are **bound** and **check**. A brute force solution without a bound is a guess. A brute force solution without an exact checker is just a loop.

3.1 Example: two independent variables

Suppose we need nonnegative integers x , y , and z with:

```
x + y + z = n
a*x + b*y + c*z = s
```

A first reaction might be a triple loop:

```
for x from 0 to n:
    for y from 0 to n:
        for z from 0 to n:
            check both equations
```

That is $O(n^3)$. If $n \leq 300$, this is about 27 million combinations per test case. It may be high in Python and unnecessary in any language.

The first equation lets us derive:

```
z = n - x - y
```

Now only x and y need to be enumerated:

```
for x from 0 to n:
    for y from 0 to n - x:
        z = n - x - y
        check cost
```

This is $O(n^2)$, about 90,000 pairs for $n = 300$. That is comfortably small.

3.2 Candidate count table

Pattern	Candidate count	Example constraint
one loop	n	$n \leq 10^7$ can be plausible
two loops	n^2	$n \leq 3000$ in C++, smaller in Python
three loops	n^3	$n \leq 300$ is already large
all subsets	2^n	$n \leq 20$ is classic
all permutations	$n!$	$n \leq 8$ or 10

This table is a starting point. The actual limit depends on constants, number of test cases, and language.

4 Exhaustive subsets

A subset of n items can be represented by an n -bit mask. Bit i is 1 if item i is chosen.

For $n = 4$, the masks are:

mask in binary	chosen indices
0000	empty subset
0001	0
0010	1
0011	0, 1
0100	2
1111	0, 1, 2, 3

The loop is:

```
for mask from 0 to  $2^n - 1$ :
    total = 0
    for i from 0 to  $n - 1$ :
        if bit i is set in mask:
            total += value[i]
    use total
```

The complexity is $O(n \cdot 2^n)$. For $n = 20$, this is about 20 million bit checks. That is fine in C++ and still reasonable in Python for a small number of cases, especially in a learning setting.

5 Simulation versus brute force

Simulation follows the timeline of the statement. Brute force explores possible answers.

Technique	You iterate over	Typical question
Simulation	commands, time steps, events	what happens?
Brute force	candidates, choices, subsets	which choice works?

Some problems combine both. You may brute force an initial choice and then simulate the process that follows from that choice.

6 Bounds before loops

Before writing nested loops, compute the largest number of iterations.

For Problem B:

```
n <= 300
number of (x, y) pairs with x + y <= n
= (n + 1)(n + 2) / 2
= 301 * 302 / 2
= 45451
```

With at most 100 test cases, the absolute worst case is about 4.5 million pairs. This is fine.

For Problem C:

```
n <= 20
subsets = 2^20 = 1048576
inner scan length = 20
rough operations = about 21 million per test
```

That is acceptable for the local exercise. In real contests, if T were large and every case had $n = 20$, you would inspect the total bound carefully. Later sections will introduce meet-in-the-middle and dynamic programming for larger subset problems.

7 Proof habits for exhaustive search

Brute force proofs are often short but must say two things:

1. **Coverage:** every valid candidate is considered.
2. **Selection:** the algorithm returns the correct candidate among those considered.

For the ticket problem:

- coverage: every nonnegative triple with $x + y + z = n$ has some x and y in the loops, and then z is derived;
- selection: the first triple passing the cost check is exactly the first valid triple in the required scan order.

For the subset problem:

- coverage: every subset corresponds to exactly one bit mask;
- selection: the algorithm keeps the maximum feasible sum seen so far.

8 Brute force as a testing oracle

Even when brute force is too slow for final constraints, it is often perfect for small random tests.

Later, for a problem with $n \leq 200000$, we might write:

```
fast solution:  $O(n \log n)$   
slow solution:  $O(n^2)$ , only for  $n \leq 30$   
random generator: many small arrays  
compare fast output to slow output
```

This is called stress testing. It is one of the most useful habits in competitive programming because it finds counterexamples to incorrect ideas.

In this section, brute force is the accepted solution. Later, it will also be your debugging partner.

9 C++ implementation notes

For simulation:

```
for (char ch : s) {  
    if (ch == 'U') ++y;  
    else if (ch == 'D') --y;  
    else if (ch == 'L') --x;  
    else ++x;  
}
```

For bounded loops:

```
for (int x = 0; x <= n; ++x) {  
    for (int y = 0; y + x <= n; ++y) {  
        int z = n - x - y;  
    }  
}
```

For subset masks with $n \leq 20$, $1 \ll n$ fits in `int`. For larger n , use `1LL << n`, but remember that enumerating 2^n is then probably too slow.

10 Python implementation notes

For simulation, strings are already iterable:

```
for ch in s:  
    ...
```

For subsets:

```
for mask in range(1 << n):  
    total = 0  
    for i, value in enumerate(values):  
        if mask & (1 << i):  
            total += value
```

This is clear and good for $n \leq 20$ in a learning exercise. Python has faster variants, but the first implementation should be correct before it is clever.

11 Common mistakes

- Simulating only the final state when the problem asks for a maximum over time.
- Updating the answer before applying the move when the statement says after.
- Writing one more nested loop than necessary.
- Forgetting that 0 is a valid count or that the empty subset may be valid.
- Using `range(n)` when the value `n` itself is allowed.
- Using a bit mask loop but accidentally starting at 1, excluding the empty subset.
- Returning any valid output when the local judge requires a deterministic first valid output.

12 Before you submit

For simulation:

1. What is the complete state?
2. Are commands processed in the exact input order?
3. Is the answer updated before or after the state change?
4. Does the starting state count?

For brute force:

1. What is the candidate space?
2. How many candidates are there in the worst case?
3. Does the loop include boundary values?
4. Is every valid answer represented?
5. If several answers are valid, does the output rule choose one deterministically?

13 Exercises

The local exercises focus on three basic forms:

1. `a_robot_walk`: direct simulation with a quantity tracked over time.
2. `b_ticket_split`: bounded enumeration with one variable derived.
3. `c_best_subset`: exhaustive subset search with bit masks.

Run:

```
python3 sections/02_simulation_bruteforce/check.py
```